DEVELOPER HANDOVER · IOS

GetDraft — everything you need to ship the iOS app

Written for someone opening this codebase for the first time. It covers what the product is, how the pieces fit, how to run it, and what App Store submission will require — including the parts that will bite you.

PLATFORM	ANDROID	IOS	BACKEND	REVISED
Expo SDK 54 · RN 0.81	shipping · v1.0.0 (29)	never built	live	13 Aug 2026

Read this before anything else

The iOS app has never been built. Not once. Never compiled, never installed, never opened. Nobody knows whether it launches. Everything shipped so far is Android.

Treat your first build as a discovery exercise, not a formality. The JavaScript is shared and healthy, but iOS-specific configuration — capabilities, entitlements, push, privacy manifest — has never been exercised.

1 · What the product is

GetDraft connects athletes with recruiters and coaches. The core interaction is a swipe deck: you see a profile card and either **Draft** it (interested) or **Pass**. When two people Draft each other it becomes a match and a conversation opens.

Vocabulary is product-specific and appears throughout the code. Keep it:

TERM	MEANING
Draft	A right swipe. Never call it “like”.
Pass	A left swipe. Never “reject” or “X”.
Super Draft	A scarce, high-priority Draft. Capped on every plan.
Draft Board	The matches screen.
Draft Score	Ranking metric behind the leaderboard.
“It’s a Draft!”	The match celebration.

Four roles, four different apps

Role is chosen at signup and decides navigation, permissions and which tabs exist. It is not cosmetic.

- **Athlete** — builds a profile, swipes, gets ranked.
- **Recruiter / Coach** — browses athletes, sends outreach.
- **Parent** — oversight only. Parents cannot swipe. A 403 there is correct behaviour, not a bug.
- **Admin** — provisioned in the database only, never self-assignable.

Minors are a first-class concern

Many athletes are under 18, and two mechanisms exist because of it. Both are load-bearing; do not route around them.

- **Guardian consent.** An under-18 athlete lands in `pending_guardian` and is blocked from feature endpoints until a parent scans their QR code, answers a questionnaire and records a consent video. Enforced server-side by `ActivationGuard`.
- **Identity verification (KYC).** Handled by Didit. Mandatory in onboarding — the only way past it is "Finish later — log out".

2 · Architecture

LAYER	STACK	WHERE
Mobile app	Expo SDK 54, React Native 0.81, expo-router 6, Redux	repo root
Backend API	NestJS 11 on Fastify, 18 modules, ~112 endpoints	`backend/`
Database	Supabase Postgres (auth, storage, realtime)	managed
ORM	Prisma 6 (Discover only) + supabase-js (everything else)	`backend/`
Hosting	Railway	managed

The app never talks to the database

Every request goes through the NestJS API, which holds the `service_role` key. The app only carries the public `anon` key, and that key has had all table privileges revoked. Do not add direct Supabase queries from the app; they will fail, and that is deliberate.

The response envelope — the one thing that will catch you

Successful responses are wrapped:

```
{ "statusCode": 200, "data": { ... } }
```

Except when the payload contains `cards`, `checkoutUrl` or `statusCode`. Those pass through raw. Reading `data.data` on one of those returns `undefined`, and you get a plausible-looking success that renders nothing. The Discover feed is the common case:

```
// services/discover.ts
return (data?.data ?? data) as FeedResponse;
```

Errors are always shaped:

```
{ "statusCode": 400, "message": "...", "error": "Bad Request", "timestamp": "..." }
```

3 · Running it locally

1. **Clone and install.** Node 20+.

   ```
   git clone https://github.com/ProgixDev/getdraft.git
   cd getdraft
   npm install
   cd backend && npm install && npx prisma generate && cd ..
   ```

   `prisma generate` is not optional — the backend will not typecheck without it.

2. Drop in the env files you were sent: `app.env` → `.env` at the repo root, `backend.env` → `backend/.env`.

3. **Run the backend** (optional — the app defaults to production).

   ```
   cd backend && npm run start:dev     # :3000
   ```

   To use it, set `EXPO_PUBLIC_API_URL=http://localhost:3000/api`.

4. **Run the app.**

   ```
   npx expo start
   ```

   Expo Go is not enough — the app uses native modules (Stripe, WebView, camera, updates). You need a development build: `eas build --profile development --platform ios`.

5. **Verify before changing anything.**

   ```
   npx tsc --noEmit                            # app     → 0 errors
   npx tsc --noEmit -p backend/tsconfig.json   # backend → 0 errors
   cd backend && npx jest                      # 90 tests → all pass
   npx expo lint                               # 0 errors, warnings only
   ```

Test login that always works

A sandboxed number, so no SMS is sent and no credit is consumed:

```
Phone: +213558780131
Code:  123456
```

Real phone numbers currently cannot receive a code — the SMS account has no balance. Email signup works normally.

4 · External services

SERVICE	USED FOR	STATE
Supabase	Database, auth, storage	Live · **FREE plan**
Railway	Backend hosting	Live
Stripe	Subscriptions, Draft packs	**LIVE keys — real money**
Prelude	Phone OTP	Working · EUR 0.00 balance
Resend	Email + Supabase auth SMTP	Live
Didit	Identity verification	Live
Mapbox	Globe tab, location search	Live
Expo / EAS	Builds, OTA updates	Live

Two service facts that will surprise you

Supabase is on the free plan. No backups at all, 1 GB of storage total (roughly 65 videos across all users, ever), and the project suspends itself after about a week of inactivity. It has already done that once: the project paused, Supabase withdrew its DNS, and the whole app went down until it was moved.

Stripe keys are live. Completing a purchase while testing charges a real card. Opening the payment sheet is free; confirming is not.

5 · iOS — what is actually required

BLOCKER

In-app purchases will fail review as-is

The app sells subscriptions and Draft packs through Stripe's native PaymentSheet — not a web link-out. That is a complete third-party purchase flow for digital goods inside the app, which is exactly what Apple requires StoreKit for. **Apple enforces this more strictly than Google.**

The switch already exists. `constants/purchases.ts` exports `PURCHASES_ENABLED`. Set it to `Platform.OS !== 'ios'` and every purchase entry point disappears — the per-plan Upgrade buttons, "Buy more Drafts", and the `/buy-swipes` route (which redirects, so a deep link cannot reach the sheet either). Plans stay visible, and cancel/resume keep working because managing a subscription is not selling one. The gating is written and was tested working on Android before being deliberately reverted. It is one line.

Also needed before submission

- **Apple Developer account** — $99/yr; identity verification takes days. `eas.json` references team `89884BGNZR`; confirm it is current and yours to use.
- **Push certificates (APNs)** — nothing configured for iOS. See §7; Android push is working, so the server side is done and this is purely the iOS credential.
- **Privacy manifest** (`PrivacyInfo.xcprivacy`) — Apple now requires it. Not present.
- **Permission strings** — already written in `app.json` under `ios.infoPlist` (camera, mic, photos, location). Review the wording; Apple rejects vague ones.
- **Sign in with Apple** — required if you ever add another social login. None today, so not yet applicable.

```
com.getdraft.app        # bundle id — matches Android, already set in app.json
```

6 · App Store submission

1. **Gate purchases off for iOS** — see the blocker above. Do this first; it changes what you submit.
2. **First build.** `eas build --platform ios --profile production`. EAS handles signing; you will be asked to sign in to Apple. Expect this to surface problems — it has never run.
3. **Install on a real device and use it.** Sign up, complete onboarding, upload a photo, open every tab. Nothing on iOS has ever been verified.
4. **Create the app in App Store Connect** with the bundle id above.
5. **Provide a reviewer account.** The app shows nothing until you sign in, so a reviewer who cannot get in rejects it. Use the sandbox phone above and add this note verbatim:

   ```
   When asked for a date of birth, please enter an adult
   date. Accounts under 18 are held for guardian approval
   by design.
   ```

   Without it, a reviewer entering a teenage birthday lands in the guardian-approval wait and concludes the app is broken.

6. **App Privacy questionnaire.** Declare honestly: name, email, phone, date of birth, precise location, photos and videos, messages, payment history, identifiers. Third-party identity verification (Didit). Mismatches here are a common rejection.
7. **Age rating.** Declare user-generated content and person-to-person messaging. Undeclared chat gets re-rated later.
8. **Screenshots** for every required device size, plus description and keywords.
9. **Submit via TestFlight first**, then production. Apps with a minor audience typically take longer.

7 · Gotchas worth knowing before you touch anything

NEW · 13 AUG

User media is served through signed URLs — never persist one

The `avatars`, `photos`, `videos` and `posts` buckets are **private**. Every API response rewrites stored media into a signed URL via a global interceptor (`SignedMediaInterceptor`), so no endpoint has to remember to do it and none can be missed. `sports` stays public — product icons, not user media.

The trap: the client receives a signed URL and can hand it straight back on the next save. Persisting it would store a token that expires, and the photo would silently vanish a week later. The same interceptor strips tokens off inbound bodies, so this is handled — but do not add a write path that bypasses it, and never store a URL containing `/object/sign/`.

Signed URLs are cached server-side and reused until near expiry. That is deliberate: `expo-image` caches by URL, so a fresh token per request would re-download every photo on every refresh. `MEDIA_SIGNING=off` disables the whole path if it ever misbehaves.

Use expo-image, never react-native's Image

React Native's `Image` decodes remote photos at full resolution. A 320 kB upload at 2048×1536 is 12 MB of bitmap; a grid of them overruns the Android heap and the process is killed. That looks like a grey screen and `ErrorBoundary` cannot catch it, because the crash is native. Twelve screens were converted for this reason.

Always capture the useFonts flag

`const [fontsLoaded] = useFonts(...)` and hold rendering until it is true. Discarding it means text is measured in the fallback face and painted in the real one — a button reading "Skip" rendered as **"Ski"** for exactly this reason. Also make sure every `fontFamily` you use is in that screen's `useFonts` call.

EXPO_PUBLIC_* must reach the cloud build

`.env` is gitignored, and EAS uses `.gitignore` to decide what to upload, so `.env` never reaches a cloud build. The Mapbox token lived only there, which meant the Globe tab and all location search were dead in *every build ever shipped* — silently, because the Globe showed a spinner rather than an error.

It now lives as an **EAS environment variable** in the `preview` and `production` environments, not in `eas.json` — GitHub's secret scanner rejects Mapbox tokens committed to the repo. Each build profile declares `environment: <name>` so EAS injects it. Verify with `eas env:list --environment production` before a release build; if it is missing, the Globe ships dead and nothing warns you.

OTA updates exist — use them, but watch the fingerprint

`expo-updates` is configured with a fingerprint runtime version and per-profile channels. JavaScript-only fixes ship in about a minute with `eas update --branch preview --platform ios`, no rebuild and no review.

Editing `eas.json` changes the fingerprint, and therefore the runtime version — an OTA then silently never reaches builds made before the edit. Check the runtime version of the installed build matches before relying on an update. The About screen prints `build <id>` or `build embedded` so you can tell which bundle a device is actually running.

There is no staging

One environment. The database, Stripe account and SMS provider you develop against are the live ones. Create throwaway accounts through the API rather than editing rows.

8 · Access you will need

The env files cover the services. These are accounts someone has to grant you:

ACCOUNT	WHY	OWNER
GitHub · ProgixDev/getdraft	The code	Progix
Apple Developer	Signing, App Store Connect	client
Expo / EAS	Builds and OTA updates	getdraftprogix
Supabase	Database, storage, auth settings	client
Stripe	Products, prices, webhooks	client
Railway	Backend env and deploys	third party — read §9
Didit / Prelude / Resend / Mapbox	Dashboards, quotas, billing	client

9 · Open risks you are inheriting

Revised 13 August 2026. Two items from the first edition are now closed; they are kept here, marked, so you can tell what changed rather than wondering whether they were missed.

RISK	STATE
Supabase free plan	**Open.** No backups, 1 GB storage, auto-suspends when idle. Has already caused one full outage.
Railway account	**Partly open.** Sits in a third party's account. Access has since been recovered, so deploys and billing are reachable — but it is still not the client's account and should be migrated.
Purchases vs store policy	**Open — and yours.** Unresolved by choice on Android; **must be resolved for iOS**. See §5.
Public storage buckets	**Closed, 13 Aug.** The four user-media buckets are private and served through signed URLs. Verified: the old public URL of a real photo now returns **400**. See §7.
Push notifications	**Android done** · **iOS open.** Delivery was never broken — the backend was discarding Expo's response, so a working setup and a broken one produced identical logs. Fixed: tickets are read, dead tokens pruned. **APNs for iOS is still unconfigured.**
Credential hygiene	**Open.** Several keys were shared over chat during setup and should be rotated. All are read from environment variables, so rotating needs no code change.

> **What is solid**
>
> The backend is verified end to end against production: all endpoints answering, KYC creating real sessions, the Stripe contract correct with no free-upgrade path, websocket auth rejecting unauthenticated sockets, uploads and MIME restrictions enforced. Nine security holes were closed and each confirmed against the live database. Onboarding completes on a real device. Typecheck is clean on both sides and **90 backend tests** pass.
>
> The shared JavaScript is in good shape. Your work is iOS-specific configuration and the purchase question — not repairing the product.